

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 2
Class Fundamentals, Inheritance
and Interface



Object Oriented Programming

- Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.
- It simplifies the software development and maintenance by providing a paradigm of:
 - **Object**
 - **Class**
 - **Inheritance**
 - **Polymorphism**
 - **Abstraction**
 - **Encapsulation**

Object

- Any entity that has state and behaviour is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

Class

- Collection of objects is called class. It is a logical entity. It is also called a blue print of your whole programming structure which contains all the members of your class.

Inheritance

- When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

- When one task is performed by different ways is known as polymorphism.
- In java, we use **method overloading** and **method overriding** to achieve polymorphism.

Abstraction

- Hiding internal details and showing functionality is known as abstraction.
- In java, we use abstract class and interface to achieve abstraction.

Encapsulation

- Binding (or wrapping) code and data together into a single unit is known as encapsulation.
- A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.

Members of Class

- 1. Variables**
- 2. Methods**
- 3. Constructors**
- 4. Blocks**
- 5. Nested Classes**
- 6. Static Members**

Inheritance

Reusability is the another aspect of OOP. It is always nice that if we can reuse something that already exist rather than creating it again.

Java support the concept of reusability by way of inheritance. Java supports four types of inheritance.

- (1) Single inheritance
- (2) Multiple inheritance (through interfaces)
- (3) Multilevel inheritance
- (4) Hierarchical inheritance

Inheritance

In java we can define a subclass using extends keyword.

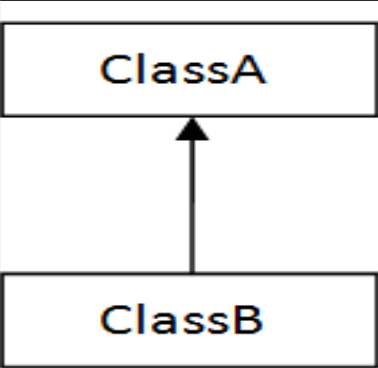
e.g.

```
class <subclass> extends <superclass>
{
    .....
    .....
}
```

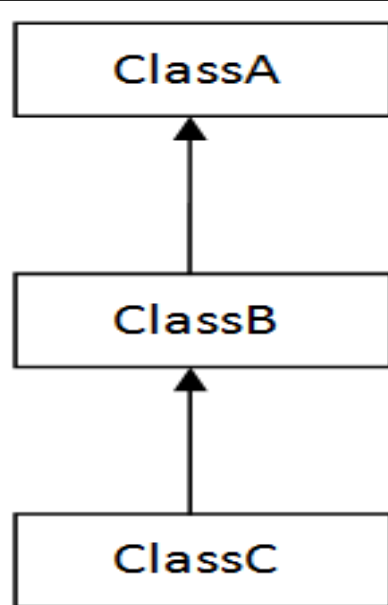
[ex/ex11.java](#)

Types Of Inheritance In Java

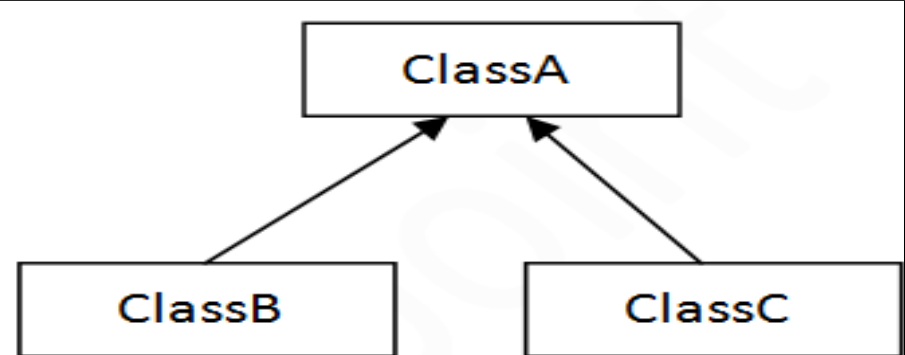
- On the basis of class, there can be three types of inheritance in java:
 - Single, Multilevel And Hierarchical.
- In java programming, Multiple and Hybrid Inheritance is supported through interface only.
- ***Note: Multiple inheritance is not supported in java through class.***
- When a class extends multiple classes i.e. known as multiple inheritance.



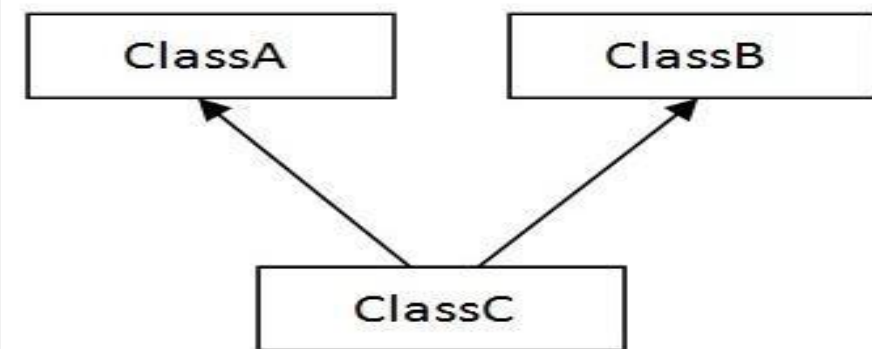
1) Single



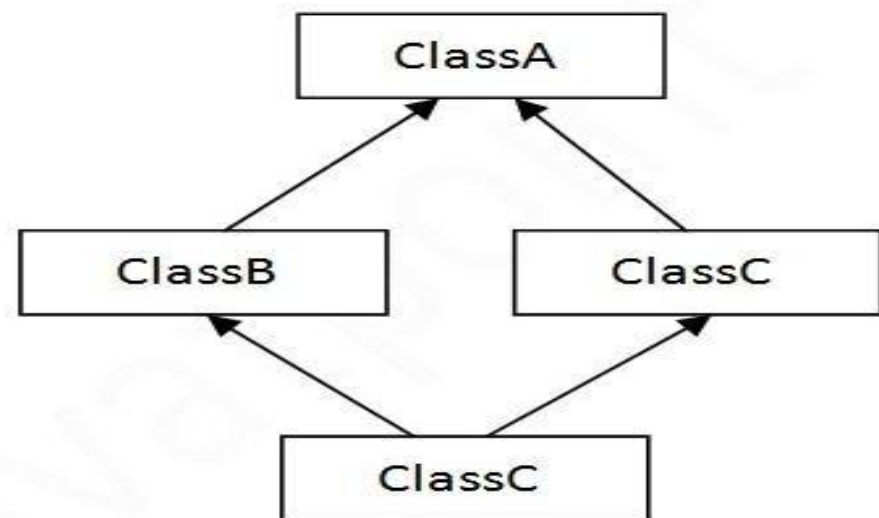
2) Multilevel



3) Hierarchical



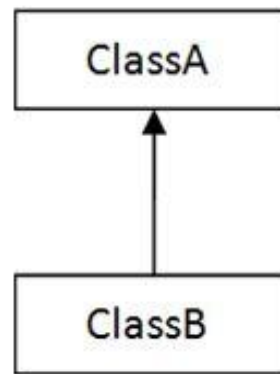
4) Multiple



5) Hybrid

(1) Single Inheritance

- In **Single Inheritance** there is only one Super Class and Only one Sub Class Means they have one to one Communication between them.
- A class inherits from **one** parent class.



1) Single

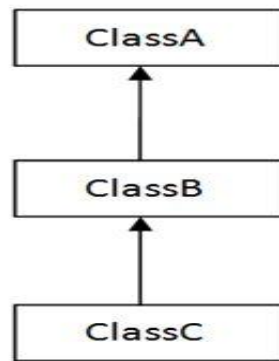
(1) Single Inheritance

```
class Animal
{
    void eat()
    {
        System.out.println("Eating...");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking...");
    }
}
```

(2) Multilevel Inheritance

- In Multilevel inheritance there is a concept of grand parent class.



2) Multilevel

- If we take the example of above diagram then class C inherits class B and class B inherits class A which means B is a parent class of C and A is a parent class of B. So in this case class C is implicitly inheriting the properties and method of class A along with B that's what is called multilevel inheritance.

(2) Multilevel Inheritance

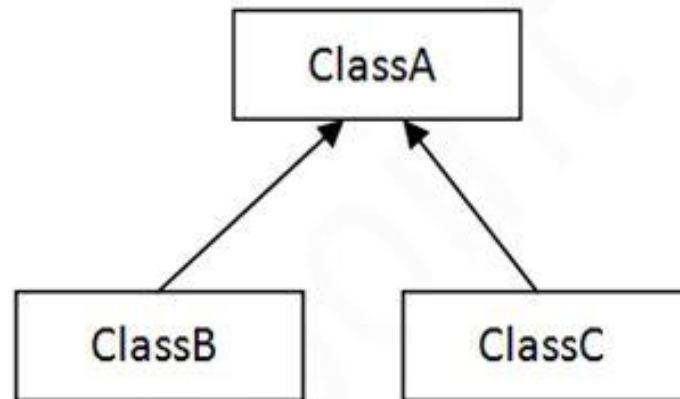
```
class Animal
{
    void eat()
    {
        System.out.println("Eating...");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking...");
    }
}

class Puppy extends Dog
{
    void sleep()
    {
        System.out.println("Sleeping...");
    }
}
```

(3) Hierarchical inheritance

- **Hierarchical Inheritance** is that in which a Base Class has Many Sub Classes or When a Base Class is used or inherited by many Sub Classes.



3) Hierarchical

(3) Hierarchical inheritance

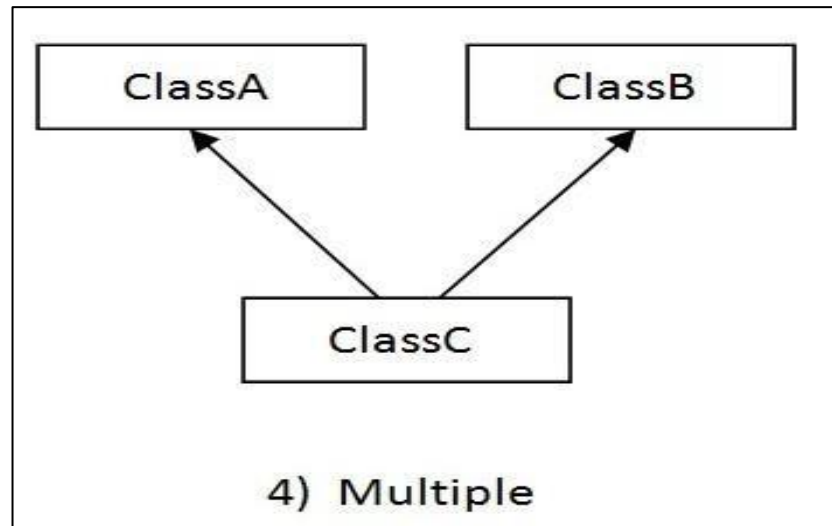
```
class Animal
{
    void eat()
    {
        System.out.println("Eating...");
    }
}
```

```
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking...");
    }
}
```

```
class Cat extends Animal
{
    void meow()
    {
        System.out.println("Meowing...");
    }
}
```


(4) Multiple Inheritance

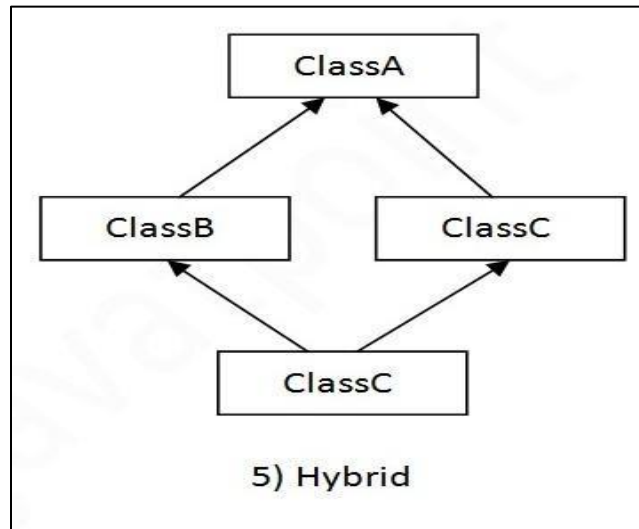
- Java **does NOT allow** a class to inherit from more than one class.
- Java supports multiple inheritance using **interfaces**.



(4) Hybrid Inheritance

- It is a **combination** of two or more inheritance types.
- Example : Hierarchical + Multilevel

But in Java, hybrid inheritance is **only partially supported** because multiple inheritance is not allowed with classes.



Method Overriding

We know that method defined in super class will be inherited in subclass. It is good that we can reuse the methods.

Sometimes it may happens that there is one method in super class and with same signature there is one method in subclass.

In **Method overriding** same name, same parameter but different class.

It is a type of **Run-time polymorphism**.

[ex\ex12.java](#)

Method Overloading

Method Overloading is a feature in Java that allows a class to have **more than one method** with the **same name** but **different parameters**.

In **Method Overloading**, same name, same class But different parameters.

It is a type of **compile-time polymorphism**.

```
class Addition {  
  
    // Method 1  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2 (different number of parameters)  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Method 3 (different type of parameters)  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Addition obj = new Addition();  
        System.out.println(obj.add(5, 10));  
        System.out.println(obj.add(5, 10, 15));  
        System.out.println(obj.add(5.5, 6.5));  
    }  
}
```

Use of final with variables and methods

All methods and variables can be overridden by default in subclass. If we wish to prevent the subclass from overriding the members of the super class then we can use final keyword.

When we use final keyword with variable name then that variable becomes constant. While using with methods it ensures that that method will never be overridden.

e.g. (1) `final int SIZE = 100;`

(2) `final void displayData();`

Use of final with classes

Sometimes we need to prevent our class to be extended. For this purpose we have to declare the class with final keyword.

e.g. final class abc

Here in above example when we declare class abc as final it will never be extended to any subclass.

finalize method

In C++ we have two special functions within the class i.e. constructor and destructor.

As we know from our earlier discussion that java does not support destructors.

So java has provides a special method called `finalize()` to free the memory occupied by non using objects.

Garbage Collector

Since objects are dynamically allocated by using the **new** operator, you might be wondering how such objects are destroyed and their memory released for later reallocation. In some languages, such as C++, dynamically allocated objects must be manually released by use of a **delete** operator. Java takes a different approach; it handles deallocation for you automatically. The technique that accomplishes this is called *garbage collection*.

Garbage Collector

It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed. There is no explicit need to destroy objects as in C++. It will not occur simply because one or more objects exist that are no longer used. Furthermore, different Java run-time implementations will take varying approaches to garbage collection, but for the most part, you should not have to think about it while writing your programs.

abstract methods and classes

When we define any class with final keyword we know that it will be never extended. Here the word abstract is totally opposite to this. When we define any class with keyword abstract at that time we must extend this class.

When we define any method with final keyword we know that it will never be overridden. Here the word abstract will work totally opposite to this. When we define any method with the work abstract then it must be override by method of subclass.

Methods with variable arguments

We can pass variable length of arguments to any method with the help of this feature. This feature is new to java and introduced from jdk1.5.

The syntax for the variable argument is as follows :

```
void <method-name> (Object...argument)
{
    ....
}
```

[ex\ex13.java](#)

Visibility Control

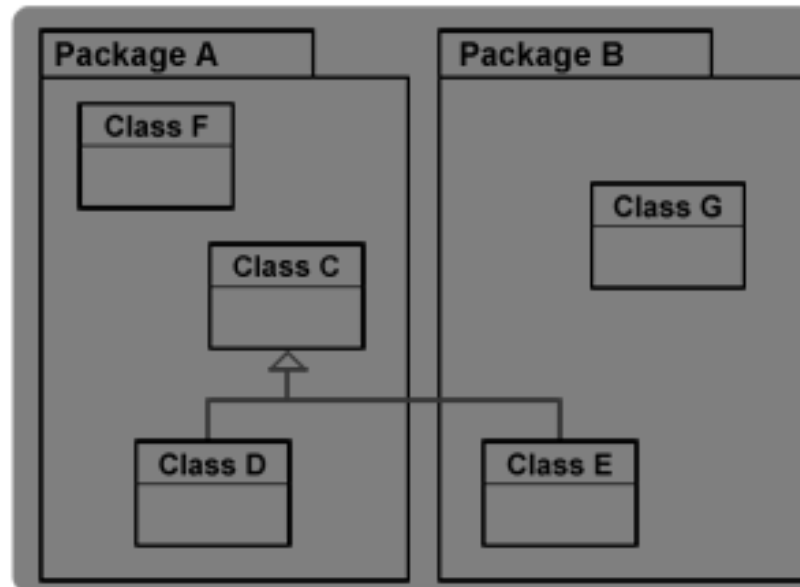
Public Access :

When we declare any variable or method as public then it is visible in the class in which it has been declared as well as in all the subclasses of this class.

To declare any variable or method as public is very risky so we need to prevent this which is possible by following level of protection.

Visibility Control

When a member of a class is modified by the public specifier then that member can be accessed by any other code in your program.



Visibility Control

Friendly access :

In java when we don't use any modifier then it gives us a limited version of public modifier which is known as friendly level of access.

The main difference between public and friendly access is when we declare any variable as public then it is visible in all classes regardless of the package while friendly access will gives visibility in the same package only.

Visibility Control

Protected access :

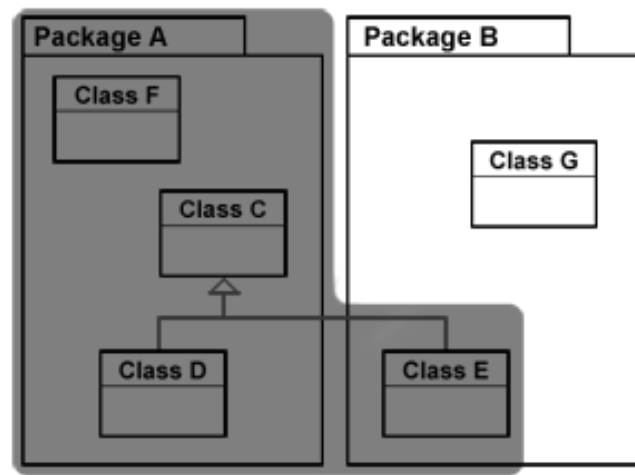
The visibility level of protected lies between public and friendly access.

Means that when we use protected then it makes fields visible not only to all the classes in the same package but also to subclasses of other packages.

Note that : non-subclass in other package cannot access the protected members.

When a member of a class is specified as protected it is available to all classes in the same package and also available to all subclasses of the class that owns the protected feature.

This access is provided even to subclasses that reside in a different package from the class that owns the protected feature.



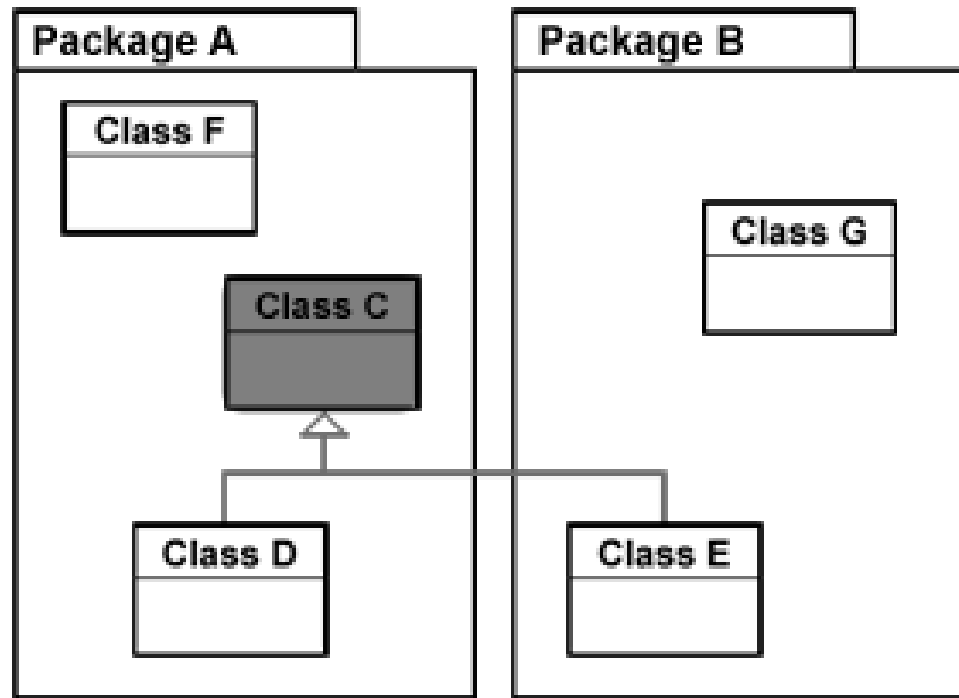
Visibility Control

Private access :

Private provides us highest degree of protection.

Means that the variable declared with private will be accessible only in the class in which it is declared so no question of inheritance.

When a member of a class is specified as private then that member can only be accessed by other members of its class.



Visibility Control

Private Protected access :

The visibility level lies between private and protected access.

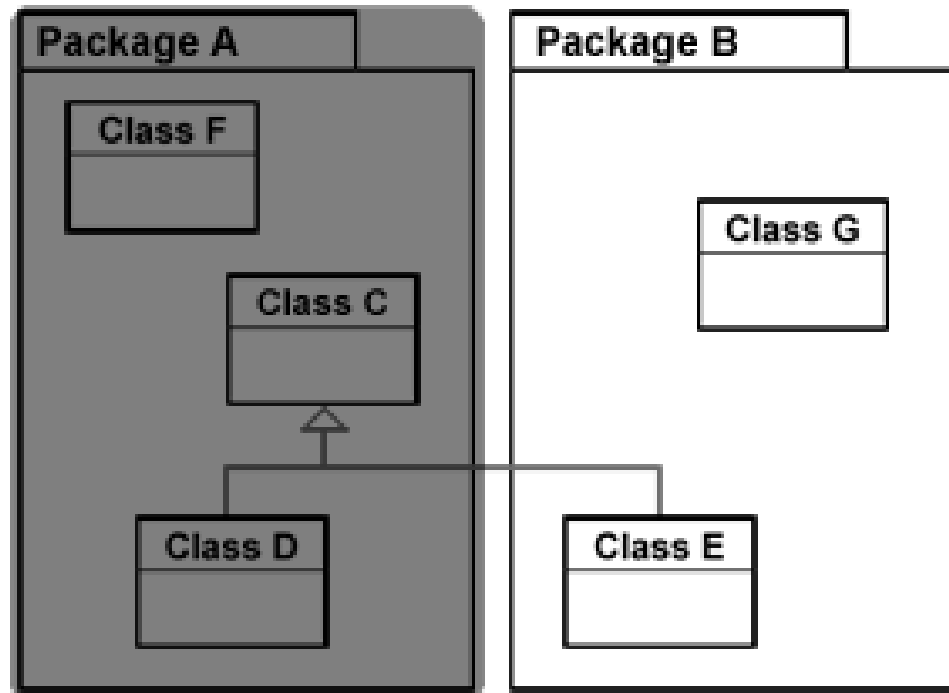
e.g. `private protected int x;`

Means that when we use private protected then it makes fields visible in all subclasses regardless of what package they are in.

Note that : these fields are not accessible by other classes of the same package.

When a member of a class is specified as private protected it is available to all subclasses of the class in the same package and to the subclasses of the other packages.

When no access specifier is used then by default the member of a class is public within it's own package but cannot be accessed outside of its package.



Visibility Control

Access Modifier and Access Location	Public	Protected	Friendly (default)	Private protected	private
Same class	Yes	Yes	Yes	Yes	Yes
Subclass in same package	Yes	Yes	Yes	Yes	No
Other classes of same package	Yes	Yes	Yes	No	No
Subclass in other package	Yes	Yes	No	Yes	No
Non-subclass of other packages	Yes	No	No	No	No

Visibility Control

Private Protected access :

- ❖ Use **public** if the field is visible everywhere
- ❖ Use **protected** if the field is to be visible everywhere in current package and subclasses of other packages.
- ❖ Use **default** if the field is to be visible everywhere in the current package only.
- ❖ Use **private protected** if the field is to be visible only in subclasses **regardless of package**.
- ❖ Use **private** if the field is **not to be visible anywhere except its own class**.

Variable hiding and shadowing

In Java, there are three kinds of variables: local variables, instance variables, and class variables. Variables have their scopes. Different kinds of variables have different scopes. A variable is **shadowed** if there is another variable with the same name that is closer in scope. In other words, referring to the variable by name will use the one closest in scope , the one in the outer scope is shadowed.

A Local Variable Shadows An Instance Variable

Inside a class method, when a local variable have the same name as one of the instance variable, the local variable shadows the instance variable inside the method block.

In the following example, there is a instance variable named "a", and inside the method setA() there is a local variable "a":

[ex\ex34.java](#)

[ex\ex35.java](#)

A Class Variable Shadows the Inherited Variable from Its Parent Classes

When an instance variable in a subclass has the same name as an instance variable in a super class, then the instance variable is chosen in the class that is the reference type.

A class can declare a variable with the same name as an inherited variable from its parent class, thus "**hiding**" or **shadowing** the inherited version. (This is like overriding, but for variables.) For example,

[ex\ex36.java](#)

Access Supper Class Variables

When shadowing does happen, you can access the super class name by either the syntax ***super.name*** or by casting the object to its super class, with the syntax ***((Superclass)object).name***.

Variables shadowed by Interface hierarchies are referenced the same way they are for class hierarchies. The following example shows the shadowing from interface hierarchies:

[ex\ex37.java](#)

[ex\ex38.java](#)

Static Keyword

- The **static** keyword in Java is used to create members (variables, methods, blocks, or nested classes) that **belong to the class**, not to any specific object.
- A static member is shared by **all objects** of the class.
- We can access static members **without creating an object**.

Static variable

```
class Student
{
    static String college = "ABC College"; // static
variable
    String name;
}
```

Static Methods:

```
class Test
{
    static void show()
    {
        System.out.println("Static method called");
    }
}
```

```
public class Main
{
    public static void main(String[] args)
    {
        Test.show(); // No object needed
    }
}
```

Static Class

```
class Outer
{
    static class Inner
    {
        void msg()
        {
            System.out.println("Static inner class");
        }
    }
}
```

Interface

Introduction

Since beginning we know that java does not support multiple Inheritance. Because we can't write like

class A extends B extends C

On the other hand we also know that multiple inheritance is also 100% useful tool. So java provided us a new approach known as interfaces.

Remember that : java class cannot be a subclass of more than one Super class but it can implement more than one interface.

Defining Interface

In general words we can say interface is basically a kind of class. Like classes in interfaces it also contains methods, variables etc. but with some difference. The main difference is in interface we can define only abstract methods and final fields.

Syntax :

```
interface <interface name>
{
    variable declaration;
    methods declaration;
}
```

Defining Interface

Example

```
interface Item
```

```
{
```

```
    static final int code=1001;
```

```
    static final String name="Fan";
```

```
    void display();
```

```
}
```

Extending Interface

Just like simple class we can extend one interface to another also. The new interface will inherit all the members of the super interface in the same manner like subclass syntax :

```
interface i2 extends i1
{
    body of i2;
}
```

Extending Interface

Example :

```
interface ItemConstants
{
    static final int code=1001;
    static final string name="Fan";
}

interface Item extends ItemConstants
{
    void display();
}
```

Implementing Interface

Interfaces are used as a super class which we can inherit in our class which is must.

syntax :

```
class <classname> implements <interfacename>
{
    body of class
}
```

Remember that : We can implement more then one interface into one class by implementing more then one interface separated by comma.

Implementing Interface

```
class <classname> extends <superclass>  
    implements <interface1>,<interface2>....  
{  
    body of the class  
}
```

[ex\ex20.java](#)

Implementing Hybrid inheritance

Interface can be used to declare a set of constants that can be used in different classes.

we can use one class to be extended and more than one interfaces to be implemented.

[ex\ex21.java](#)



Thank You